

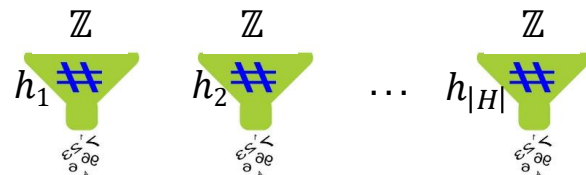
Locality-sensitive Hashing I

COMPSCI 753

Zijian Huang

The University of Auckland

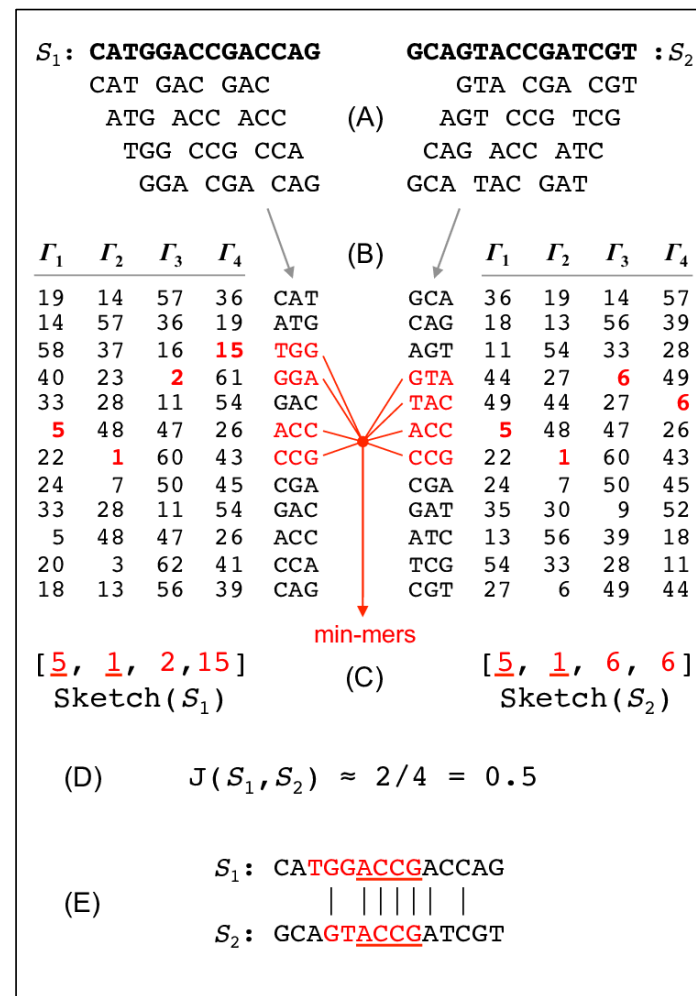
RECAP: Universal Hashing



- **Universal hashing:** Given a random choice of a hash function h from the universal family such that $h: U \rightarrow \{0, 1, \dots, n - 1\}$
 - $h(x)$ spreads the data items out over the range of possible hash values
 - $h(x)$ takes less storage than the original data U
- Desirable Performance
 - **$O(n)$ space** and **$O(1)$ expected time** per dictionary operation (**insert, delete, search**)
- Expectation is due to the randomness of selection of hash functions

Outline

- Similarity Search Applications
- Finding Similar Documents
 - Shingling
 - MinHash
 - Locality-sensitive hashing (LSH)

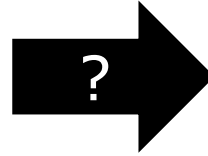
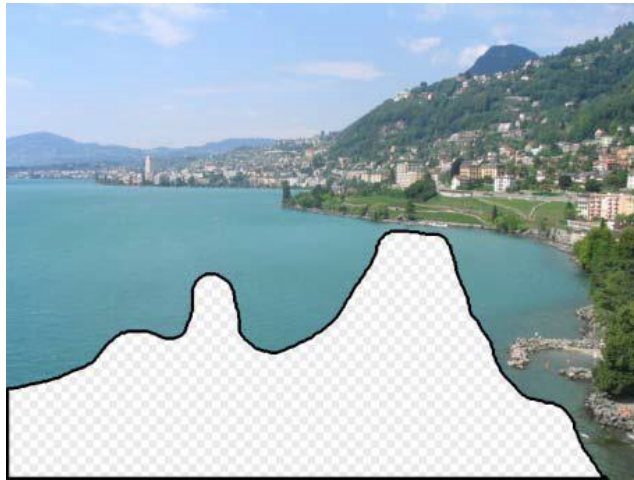


Similarity Search Problem

- **Find near-neighbors in high-dimensional space.** Many real-world problems can be expressed as finding “similar” sets
- Application Examples
 - Pages with similar words: For duplicate detection, classification by topic
 - Customers who purchased similar products
 - Images with similar features

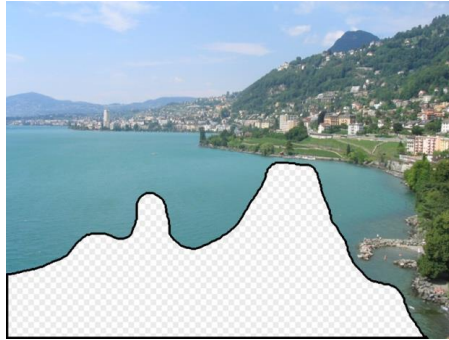
Example: Scene Completion Problem

- Problem: Patch up holes in an image by finding similar image region from a huge image database on the Web

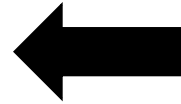


Source: <http://graphics.cs.cmu.edu/projects/scene-completion/>

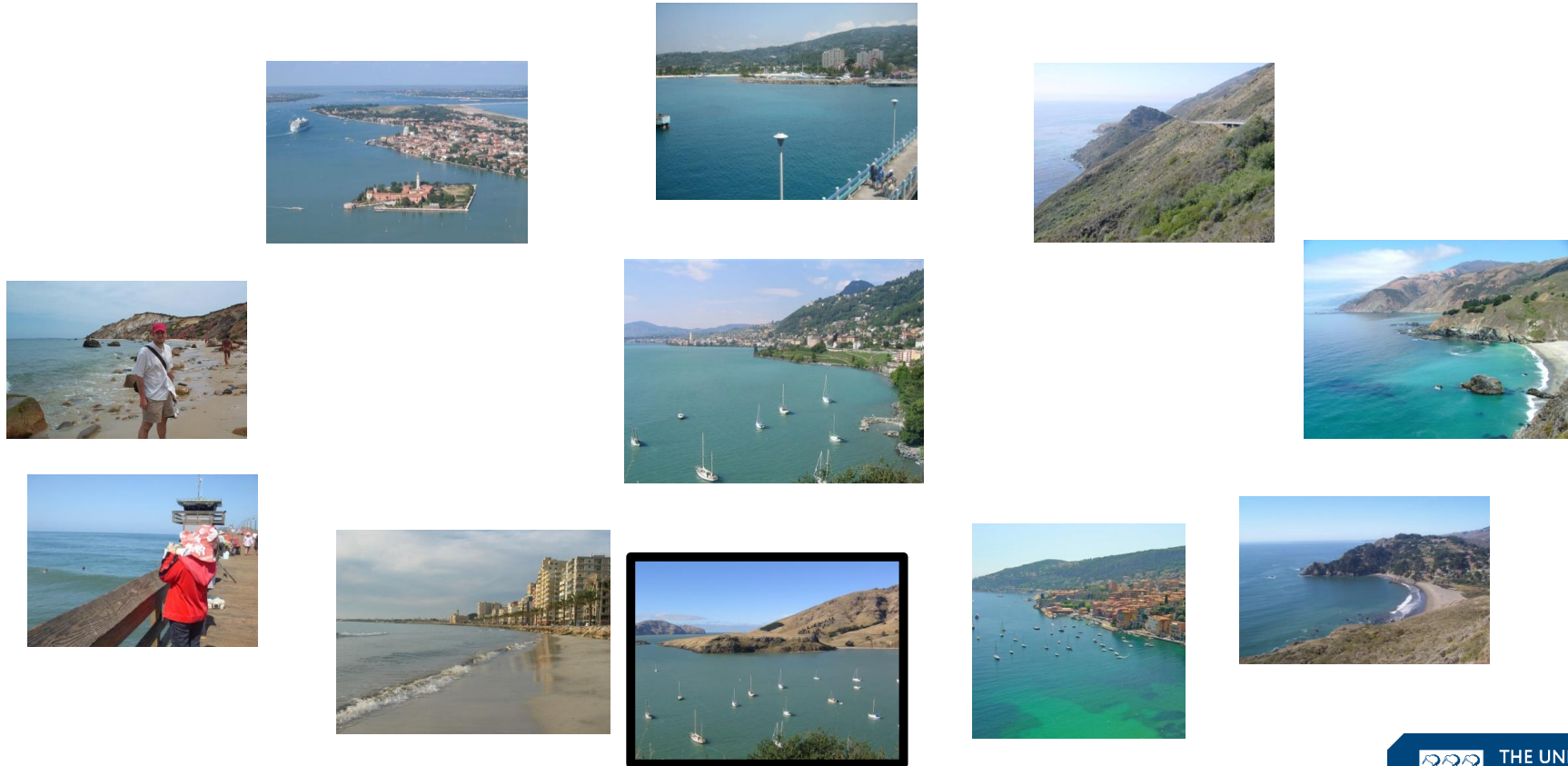
Example: Scene Completion Problem



Similarity Search

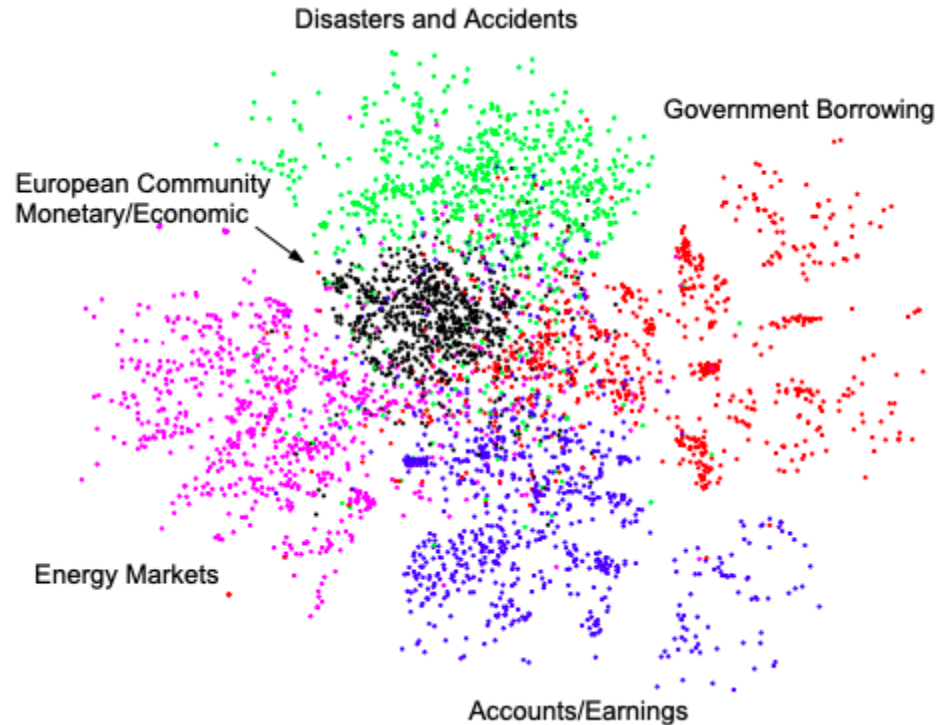


Example: Similarity Search



10 nearest neighbors from a collection of 2 million images

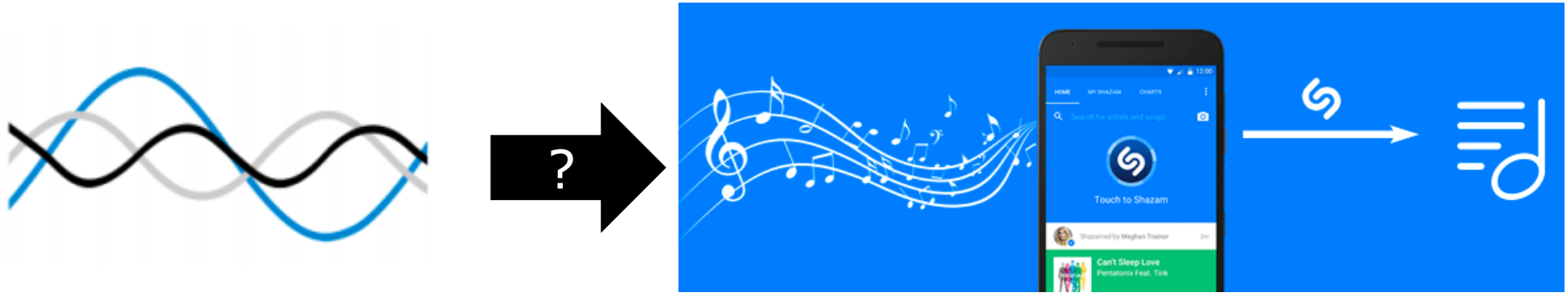
Example: News Classification



- 804,414 newswire stories
- 103 topics

Source: <http://trec.nist.gov/data/reuters/reuters.html> (Reuter Corpus)

Example: Music Recognition



**...wondering which song
I'm listening to...**

Just Shazam it...

Avery Li-Chun Wang. "An Industrial-Strength Audio Search Algorithm." ISMIR, 2003

General Problem Formulation: Similarity Search

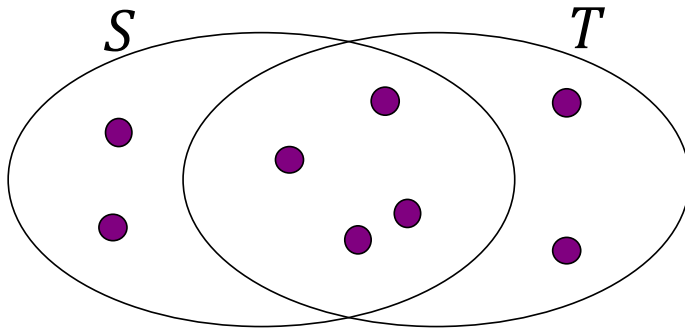
- Input
 - A data set X of n points in high-dimensional space
 - A similarity function ***sim***($\cdot$) and a similarity threshold s
- Goal: Find **all similar pairs** x_i, x_j in X such that $\text{sim}(x_i, x_j) \geq s$
- Solutions:
 - Naïve solution: $O(n^2)$ 😞
 - Magic: This be done in $O(n)$ 😊

Jaccard Similarity

- The Jaccard similarity of two sets S and T is the ratio of their intersection size to their union size

$$J(S, T) = \frac{|S \cap T|}{|S \cup T|}$$

不是size和



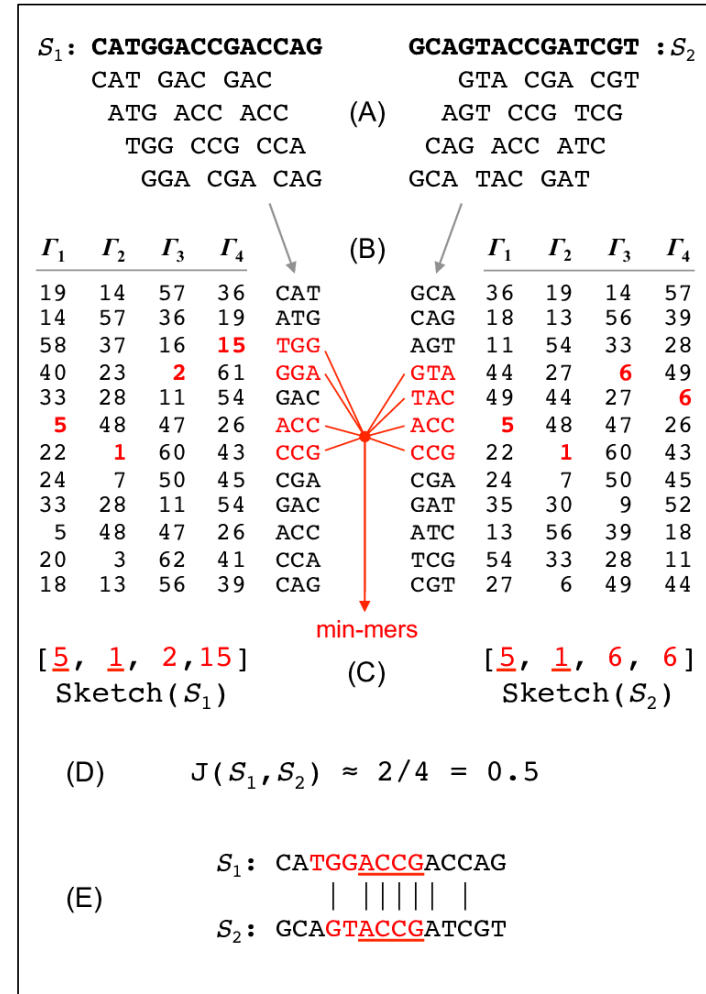
- 4 in intersection
- 8 in union
- Jaccard similarity $J(S, T) = 1/2$

Finding Similar Documents

- Input
 - n documents (e.g., web pages)
 - The Jaccard similarity function J and a similarity threshold s
- Goal: Find all near-duplicate pairs S, T such that $J(S, T) \geq s$
- Challenges
 - Many small pieces of a document (e.g., words, phrases...) can appear in different order
 - Memory constraint: n documents are too large to fit into main memory
 - Computation bottleneck: $O(n^2)$ pairs of documents to compare

Outline

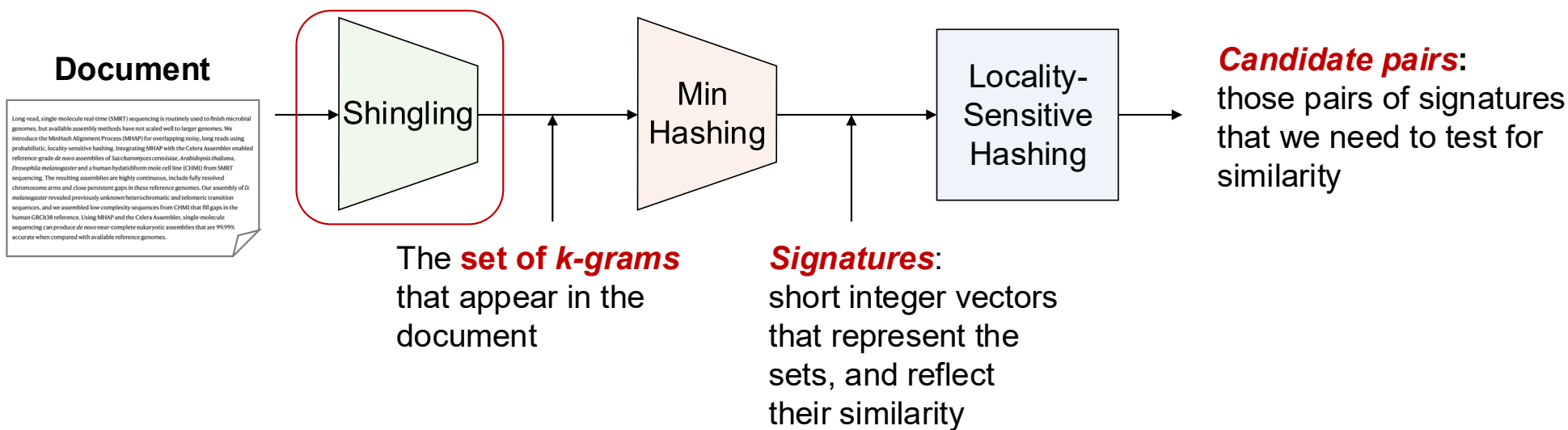
- Similarity Search Applications
- Finding Similar Documents
 - Shingling
 - MinHash
 - Locality-sensitive hashing (LSH)



Framework Overview

1. Shingling: To convert documents to **sets**
2. MinHash
 - Convert large sets to short **signatures**, while preserving Jaccard similarity
3. Locality-sensitive hashing (LSH)
 - Focus on **candidate pairs** which are likely to be similar, breaking the barrier of $O(n^2)$

Framework Overview



Web Pages as High-Dimensional Data

- Motivation
 - Doc(A): *A rose is red, a rose is white.*
 - Doc(B): *A rose is white, a rose is red.*
 - Doc(C): *A rose is a rose is a rose.*
- Simple approaches
 - Document = set of words in document
 - Document = set of “important” words in document
- Not work well: Consider the **order of words**

tf-idf not consider the order.

1. Shingling: Convert documents to sets

- k -shingle (k -gram) is a **sequence of k tokens** (e.g., characters, words)
- Represent a document as a set of k -shingles

- Example with $k = 3$:

一次取三个 按照第一个词的顺序

Doc(A): A rose is red, a rose is white. → Set(A) = {a rose is, rose is red, is red a, red a rose, rose is white}

Doc(B): A rose is white, a rose is red. → Set(B) = {a rose is, rose is white, is white a, white a rose, rose is red}

Doc(C): A rose is a rose is a rose. → Set(C) = {a rose is, rose is a, is a rose}

Example: Universal Hashing

- A hash function h maps a universe of size n (number of k -shingles) to n integers.

$$h: U \rightarrow [1,2,3,4,5,6,7,8,9]$$

- Dictionary insertions

k-shingles	Hash Value
"a rose is"	1
"white a rose"	2
"rose is red"	3
"rose is a"	4
"red a rose"	5
"is a rose"	6
"rose is white"	7
"is red a"	8
"is white a"	9

1. Shingling: Convert documents to sets

- Compressing shingles: Assign shingles into 4-byte integers
- Represent a document as a set of hash values := integers of its k -shingles
- Example with $k = 3$:

Set(A) = {*a rose is, rose is red, is red a, red a rose, rose is white*}. $\rightarrow A' = \{1, 3, 8, 5, 7\}$

Set(B) = {*a rose is, rose is white, is white a, white a rose, rose is red*}. $\rightarrow B' = \{1, 7, 9, 2, 3\}$

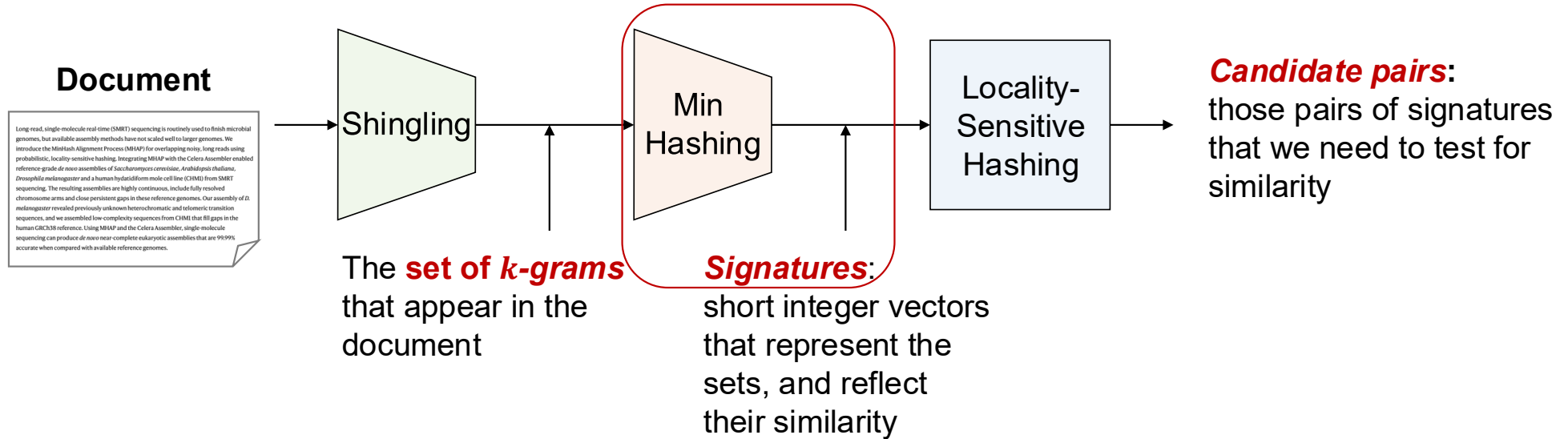
Set(C) = {*a rose is, rose is a, is a rose*}. $\rightarrow C' = \{1, 4, 6\}$

k-shingles	Hash Value
"a rose is"	1
"white a rose"	2
"rose is red"	3
"rose is a"	4
"red a rose"	5
"is a rose"	6
"rose is white"	7
"is red a"	8
"is white a"	9

1. Shingling: Convert documents to sets

- **Assumption.** If documents are similar, they will share many common shingles → have a high Jaccard similarity
- Caveat: we should pick large enough k to differentiate documents
 - $k = 5$ for short documents
 - $k = 10$ for long documents

Framework Overview



2. MinHash: Encoding a set as binary vectors

- Ideal: Encode a set into a binary vector
 - Each dimension is an element in the universal set.
- Example: Universal set $U = \{1, 2, 3, 4, 5, 6, 7\}$
 - $D_1 = \{1, 2, 5, 6, 7\} \rightarrow D_1 = 1100111$ 标识, 如果d存在这个数 标识为1 不存在
 - $D_2 = \{1, 2, 3, 6\} \rightarrow D_2 = 1110010$ 为0
 - $D_3 = \{1, 6, 7\} \rightarrow D_3 = 1000011$

2. MinHash: Encoding a set as binary vectors

- From Sets to Binary Matrix

$$D_1 = \{1, 2, 5, 6, 7\}$$

$$D_2 = \{1, 2, 3, 6\}$$

$$D_3 = \{1, 6, 7\}$$

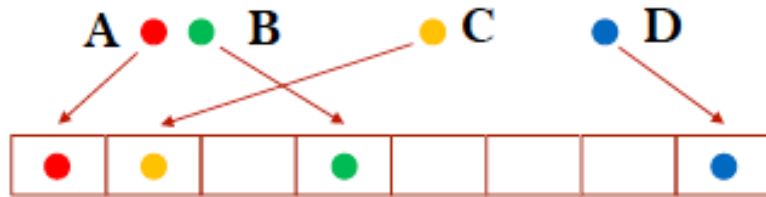
$$D_4 = \{2, 3, 4, 5\}$$



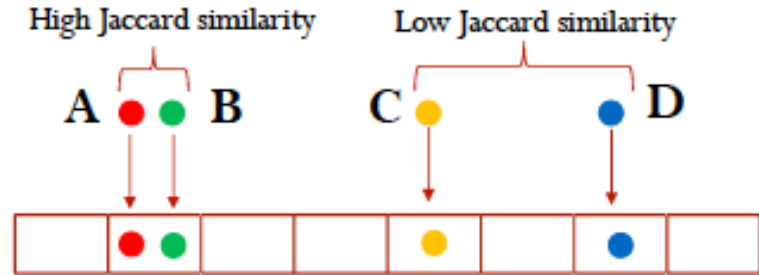
	D_1	D_2	D_3	D_4	
1	1	1	1	0	Shingles
2	1	1	0	1	
3	0	1	0	1	
4	0	0	0	1	
5	1	0	0	1	
6	1	1	1	0	
7	1	0	1	0	
Documents					

2. MinHash

- MinHash each set/document into a hash table
- Only compute Jaccard similarity of any pair of sets in the same bucket



General Hashing



Min-Hashing

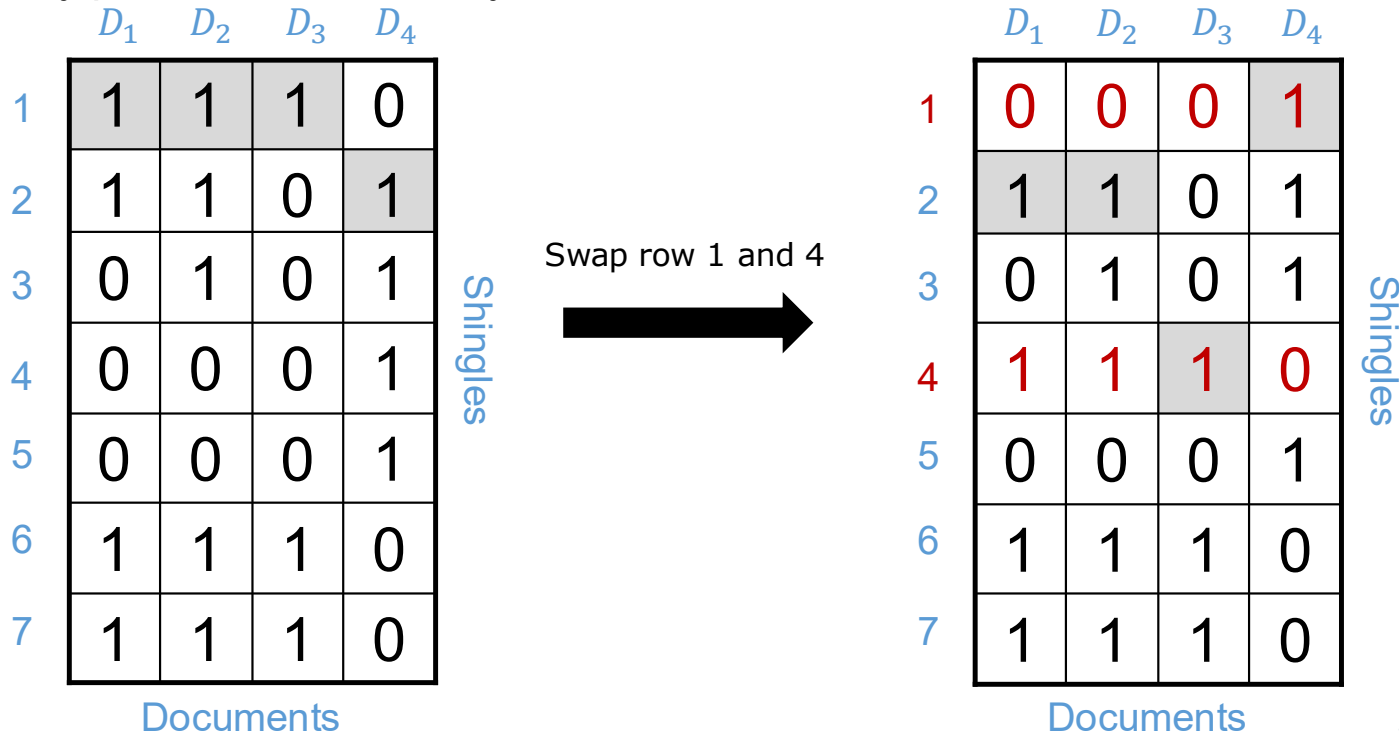
2. MinHash

- **Goal:** find a hash function h such that
 - If $J(S, T)$ is high $\rightarrow$ with high probability $h(S) = h(T)$ (collision)
 - If $J(S, T)$ is low $\rightarrow$ with high probability $h(S) \neq h(T)$ (no collision)
- **MinHash:** $\Pr[h(S) = h(T)] = J(S, T)$
- **Key steps:**
 - Perform a **random permutation** π on the rows of the Boolean matrix
 - MinHash function $h_\pi(S)$
 - The **index of the first row with value 1** of S in permuted order π :
$$h_\pi(S) = \min_{S_\pi[i]=1} i$$
 - Use several independent MinHash (i.e., permutation π) to construct the signature of each document (column)

	D_1	D_2	D_3	D_4	
1	1	1	1	0	Shingles
2	1	1	0	1	
3	0	1	0	1	
4	0	0	0	1	
5	1	0	0	1	
6	1	1	1	0	
7	1	0	1	0	
Documents					

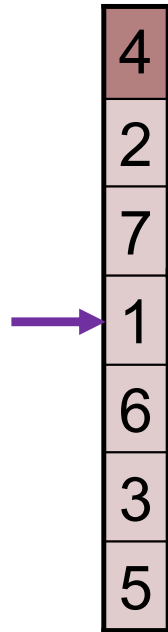
2. MinHash: Signature Generation

- Motivation:** If two documents are near duplicate, (e.g., D_1 and D_2) their first “1”s in any permutation are likely to be in the same row



Example: MinHash

Permutation π



4
2
7
1
6
3
5

	D_1	D_2	D_3	D_4
1	1	1	1	0
2	1	1	0	1
3	0	1	0	1
4	0	0	0	1
5	1	0	0	1
6	1	1	1	0
7	1	0	1	0

Shingles

Documents



Signature matrix M

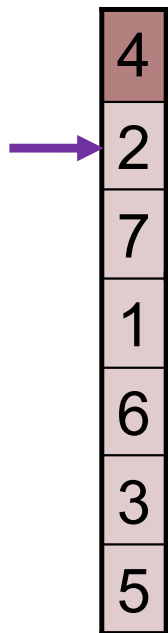
D_1	D_2	D_3	D_4
			1



1-st element of the permutation is the first to map to a 1

Example: MinHash

Permutation π



4
2
7
1
6
3
5

	D_1	D_2	D_3	D_4
1	1	1	1	0
2	1	1	0	1
3	0	1	0	1
4	0	0	0	1
5	1	0	0	1
6	1	1	1	0
7	1	0	1	0

Shingles

Documents

Signature matrix M

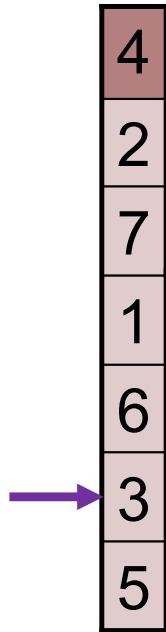
D_1	D_2	D_3	D_4
2	2		1

在第几行中出现第一次对应的在集合中

2-nd element of the permutation is the first to map to a 1

Example: MinHash

Permutation π



4
2
7
1
6
3
5

	D_1	D_2	D_3	D_4
1	1	1	1	0
2	1	1	0	1
3	0	1	0	1
4	0	0	0	1
5	1	0	0	1
6	1	1	1	0
7	1	0	1	0

Shingles

Documents



Signature matrix M

D_1	D_2	D_3	D_4
2	2	3	1

3-rd element of the permutation is the first to map to a 1

Example: MinHash

Permutation π

4	3
2	4
7	7
1	2
6	6
3	1
5	5

	D_1	D_2	D_3	D_4
1	1	1	1	0
2	1	1	0	1
3	0	1	0	1
4	0	0	0	1
5	1	0	0	1
6	1	1	1	0
7	1	0	1	0
Documents				

Shingles



Signature matrix M

D_1	D_2	D_3	D_4
2	2	3	1
1	1	1	2

Example: MinHash

只关注1的行 从中找到随机列中最小的是
1的行

Permutation π

4	3	2
2	4	3
7	7	7
1	2	6
6	6	1
3	1	5
5	5	4

	D_1	D_2	D_3	D_4
1	1	1	1	0
2	1	1	0	1
3	0	1	0	1
4	0	0	0	1
5	1	0	0	1
6	1	1	1	0
7	1	0	1	0

Shingles

Documents



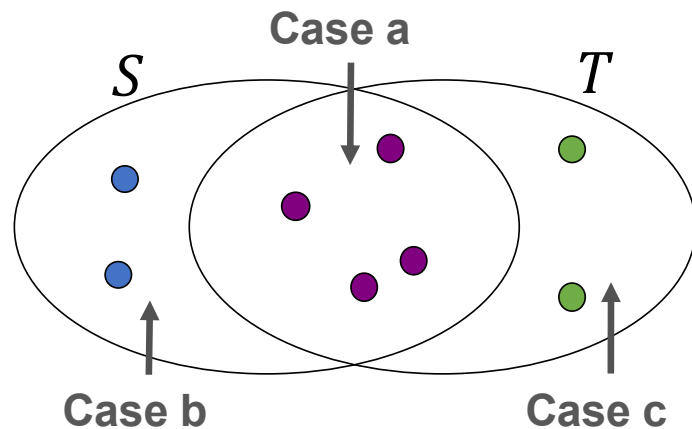
Signature matrix M

D_1	D_2	D_3	D_4
2	2	3	1
1	1	1	2
1	2	2	1

2. MinHash: Similarity of Signatures

- Claim: $\Pr[h(S) = h(T)] = J(S, T)$
- Observation in each row of the Boolean matrix:

	S	T
Case a	1	1
Case b	1	0
Case c	0	1
Case d	0	0



- For each row:
 - If we observe case a before b or c: $h_{\pi}(S) = h_{\pi}(T)$
 - If we observe case b or case c before a: $h_{\pi}(S) \neq h_{\pi}(T)$
 - $\Pr[h_{\pi}(S) = h_{\pi}(T)] = (\text{case a}) / (\text{case a} + \text{case b} + \text{case c}) = J(S, T)$

2. MinHash: Similarity of Signatures

- MinHash property: $\Pr [h_{\pi}(S) = h_{\pi}(T)] = J(S, T)$
 - Use d random permutations to obtain a signature of size d for each set
 - Given two signatures, their similarity is equivalently the fraction of MinHash functions in which they agree or collide (i.e., the number of collisions / d)
- The Jaccard similarity of two sets (columns) equals to the **expected similarity of their signatures**
 - **Proof.**

Let random variable $X_i = 1$ if the sets collide in the i -th hash function, otherwise 0

$$\mathbf{E} [\text{the number of collisions}/d] = \frac{\mathbf{E}[\sum_{1 \leq i \leq d} X_i]}{d} = \frac{1}{d} \sum_{1 \leq i \leq d} \mathbf{E}[X_i] = J(S, T)$$

Example: Similarity of Signature

Permutation π

4	3	2
2	4	3
7	7	7
1	2	6
6	6	1
3	1	5
5	5	4

	D_1	D_2	D_3	D_4
1	1	1	1	0
2	1	1	0	1
3	0	1	0	1
4	0	0	0	1
5	1	0	0	1
6	1	1	1	0
7	1	0	1	0

Documents

Shingles



Signature matrix M

D_1	D_2	D_3	D_4
2	2	3	1
1	1	1	2
1	2	2	1

Similarity $J(S, T)$	(D_1, D_2)	(D_3, D_4)
Binary Vector	$3/6=0.5$	$0/7=0$
Signature	$2/3=0.67$	$0/3=0$

6是奇数, 西(有)33

2. MinHash: Signatures Representation

- Given d random permutations, each document can be represented as a d -dimensional integer vector
 - Preserve the pairwise Jaccard similarity
 - The size of a signature is small (i.e., 100 permutations)
- Empirically, implementation of row permutation is computationally expensive
 - One-pass MinHash signatures
 - **[Book]** Chapter 3.3.5 “Mining of Massive Datasets”

One-pass MinHash Signatures

- Goal
 - Simulate the effect of a random permutation by a random hash function h such that h map integers $0, 1, \dots, n - 1$ to bucket numbers 0 through $n - 1$ with few collisions
- Idea
 - Instead of selecting d random permutations of rows
 - We select d randomly chosen universal hash functions $h_1, h_2, \dots, h_d$ on the rows
- Question: How to select a universal hash function $h(x)$?
 - $h_{a,b}(x) = ((ax + b) \bmod p) \bmod n$, where
 - $0 \leq a, b < p$ and p is a large prime $> n$

Example: One-pass MinHash Signatures

- Compute MinHash signatures using the two hash functions
 - $h_1(x) = (x + 1) \% 5$
 - $h_2(x) = (3x + 1) \% 5$

再从随机置换中计算签名
找到某列所有1的行
1的行中最小的哈希填入表

Simulated Permutation π

Row	S_1	S_2	S_3	S_4	$(x + 1) \% 5$	$(3x + 1) \% 5$
0	1	0	0	1	1	1
1	0	0	1	0	2	4
2	0	1	0	1	3	2
3	1	0	1	1	4	0
4	0	0	1	0	0	3

随机置换

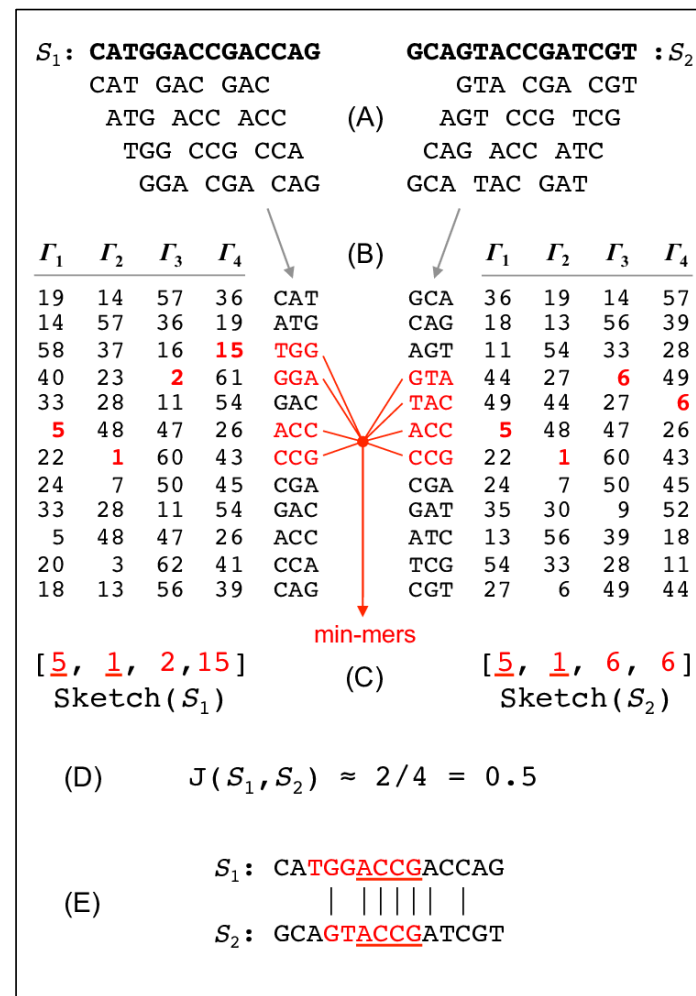
Simulated Signature Matrix

	S_1	S_2	S_3	S_4
h_1	1	0	0	1
h_2	0	2	0	0

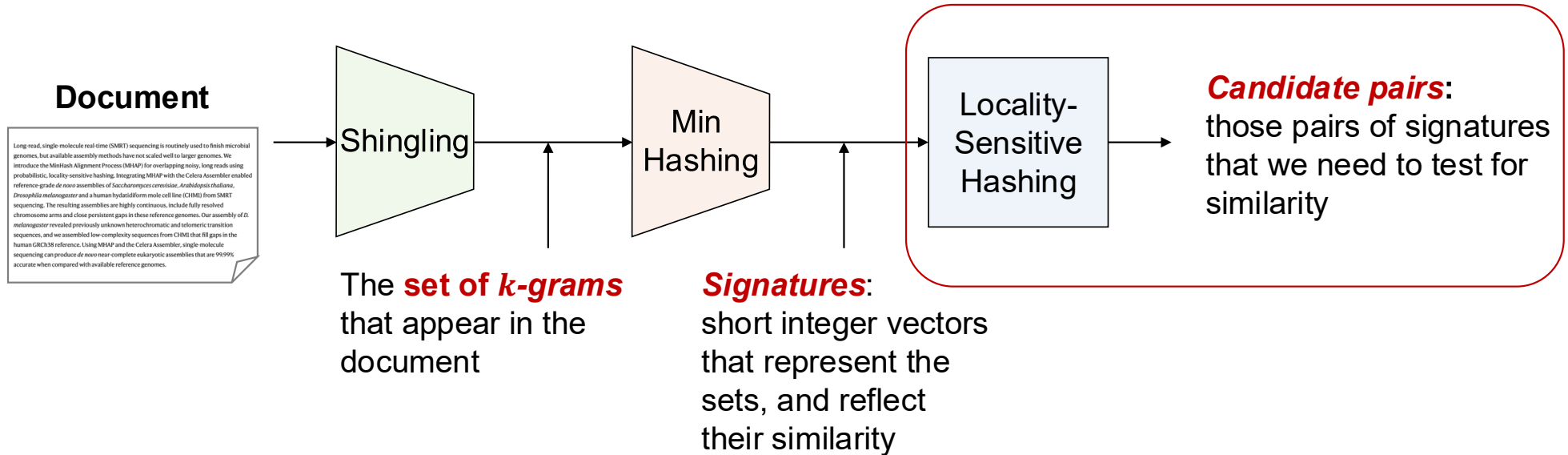
$\min(1,4) \quad \min(0,1)$

Summary

- **Shingling:** Convert documents to sets
 - We used hashing to assign each shingle an integer ID
- **Min-Hashing:** Convert large sets to short signatures, while preserving Jaccard similarity
 - We used similarity preserving hashing to generate signatures with property $\Pr[h_\pi(S) = h_\pi(T)] = J(S, T)$
 - We used hashing to get around generating random permutations efficiently



Framework Overview



Resources

- **[Book]** Chapter 13 “Algorithm Design” by Jon Kleinberg and Éva Tardos.
 - <https://www.cs.princeton.edu/~wayne/kleinberg-tardos/>
 - <https://ebookcentral.proquest.com/lib/auckland/detail.action?docID=5175682#>
- **[Book]** Chapter 3 “Mining of Massive Datasets” by Jure Leskovec, Anand Rajaraman, and Jeff Ullman.
 - <http://mmds.org>